Network Scanner

Problem statement and background:

Given a set of network traffic logs with features describing connection metadata and behavior, predict whether each connection instance represents a normal (benign) event or a malicious (attack) event — and ideally identify the type of attack.

With the exponential growth of internet-connected systems, cyberattacks have become more sophisticated and frequent. Network administrators need automated and intelligent systems to detect and prevent malicious activity in real time. Traditional intrusion detection systems (IDS) rely heavily on static rules and signatures, which cannot keep up with new or evolving attack patterns.

Machine learning offers a dynamic alternative: by training models on labeled network traffic data, we can detect patterns that distinguish normal traffic from various types of attacks.

The CIC-UNSW-NB15 dataset, created by the University of New South Wales and the Canadian Institute for Cybersecurity, provides a realistic and modern dataset for evaluating intrusion detection systems.

Data Wrangling:

The raw dataset from the Canadian Institute for Cybersecurity has 447,915 rows and 83 columns. This is too much data to try and process all at once and out of scope for this project. Some of the data includes redundant pieces of data, but good news there is no missing data from this set. The zeros in the data are just as important as the numeric data. There will be ten different types of network scenarios that I will be testing for.

These are as follows, Benign or normal activity, Analysis or gathering information about the network, Backdoor or ways that attackers will maintain access to the

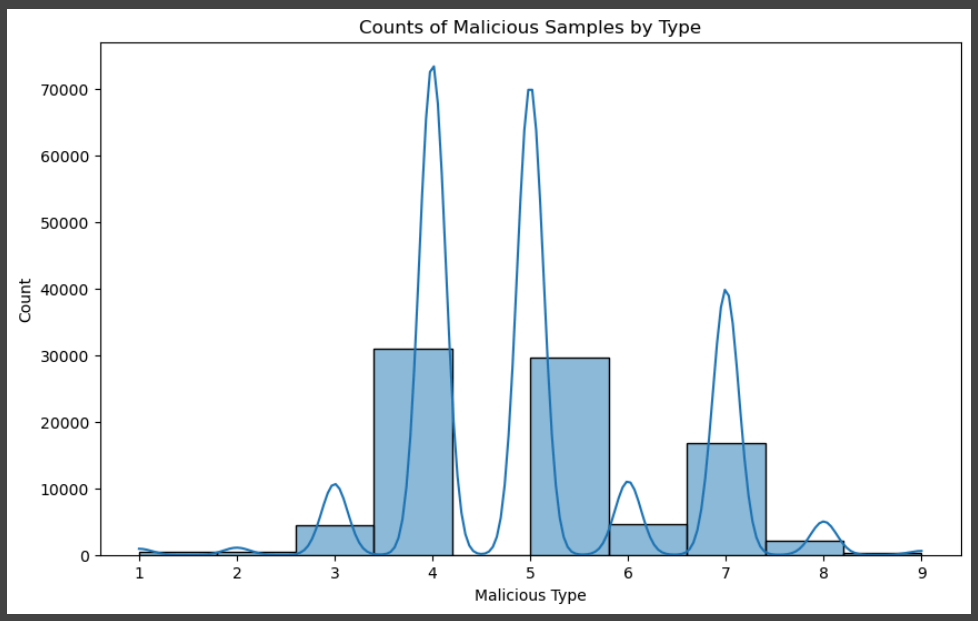
network, Denial of Service Attacks also known as DoS (this attack floods the network with transaction slowing or completely overwhelming the network causing the system to stop work as intended), and Exploits is a huge category due to the diversity of types of attacks, We will cover this a little more in depth later. For the remaining types of attacks, Fuzzers, Generic, Reconnaissance, Shellcode, and Worms.

The types of Attacks:

- **Analysis:** This isn't really an attack most of the time, but could lead to one. When looking at analysis, or network scans, most attackers are looking for vectors that are able to attack. Scanning Ports, Sniffing traffic, or overall just monitoring the system.
- **Backdoor:** This happens after an attack normally, someone got in and left themselves a door to return to.
- **DoS:** This attack floods the network with transaction slowing or completely overwhelming the network causing the system to stop work as intended
- **Exploits:** These are hardware dependent attacks, meaning this attack only works with X device with Y version of firmware. And the same type of attack won't work on X device with Z version of firmware.
- **Fuzzers:** An automated attack technique that floods systems with malformed, unexpected, or random data packets to expose security vulnerabilities, crashes, or memory leaks.
- **Generic:** Unauthorized attempts to disrupt, steal data from, or gain access to IT infrastructure. Example: Employee plugs an unidentified usb into their work PC, this usb has malware on it and now the attack was successful.
- **Reconnaissance:** These are active scans to the network usually more targeted than Analysis.
- **Shellcode:** An Attacker has found a way to execute shell code to the system gaining control. Rare but can happen.
- **Worms:** A type of self-replicating malware that spreads autonomously across networks by exploiting software vulnerabilities, without requiring user interaction

EDA:

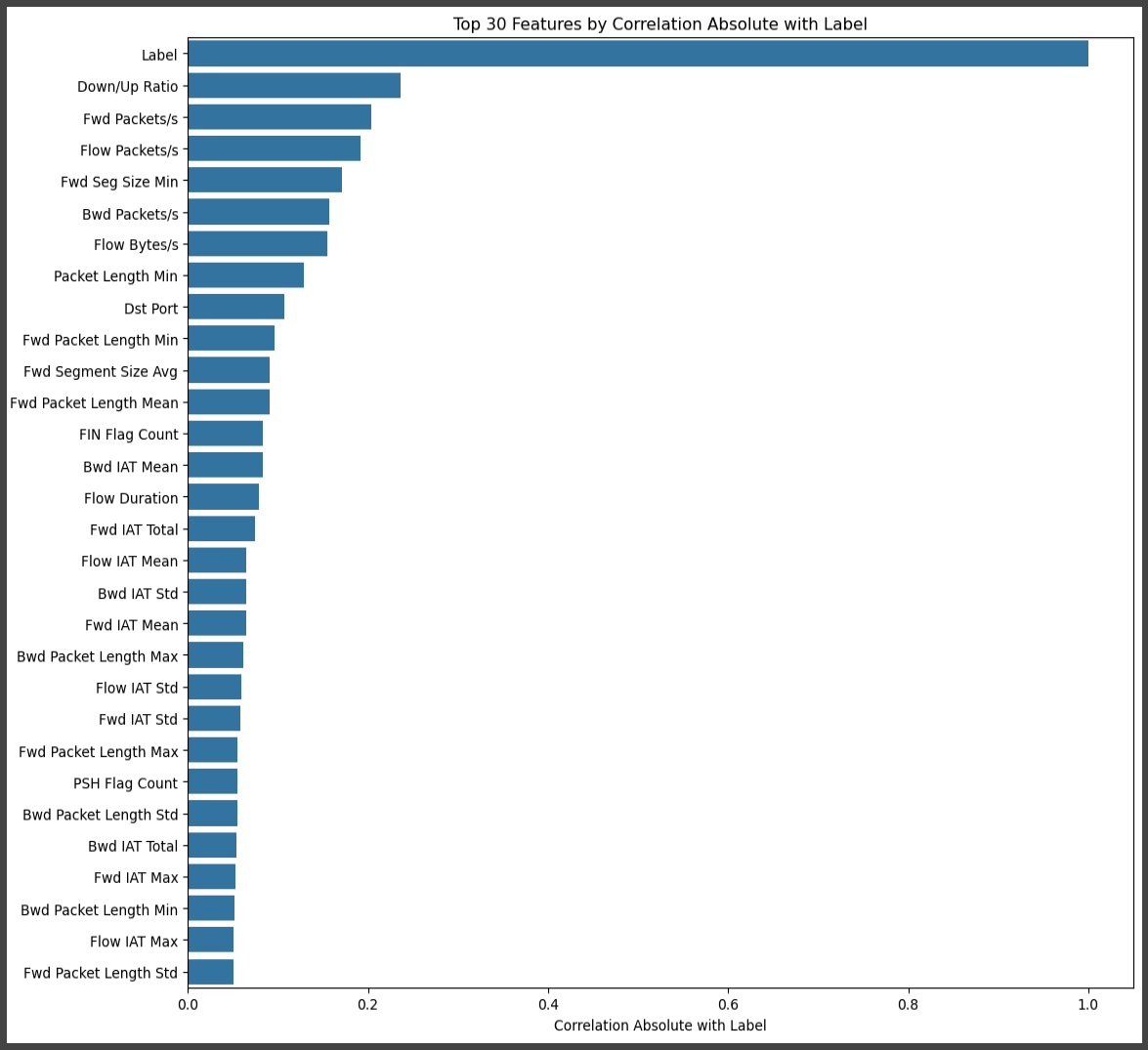
Taking a closer look at the data, there are several columns that have little to no variety. Also, this data set is too big to realistically model and process for the computing power I have access to. So sampling the data set well maintains the distribution of the data set (below, distribution of attack types without benign data due to the huge skew of the data).



After the sampling 100,000 is done the distribution of Attack types look like:

Benign	97,426
Analysis	11
Backdoor	12
DoS	128
Exploits	889
Fuzzers	850
Generic	133
Reconnaissance	480
Shellcode	60
Worms	7

Now let's talk about the correlation between each parameter compared to the label. Correlation is a percentage that determines how much one parameter affects the outcome in this cause the label or attack type. In the graph below, we can determine that there is no real strong connection between the network data, and an attack type. Meaning there is no single param we can test to determine if we are under attack on the network or not. Good news for us is that we can use multiple parameters to detect an attack. Another way we will try to detect an ongoing attack is through outliers, but to network activity varying between task to task we might get some false positives when detecting threats.



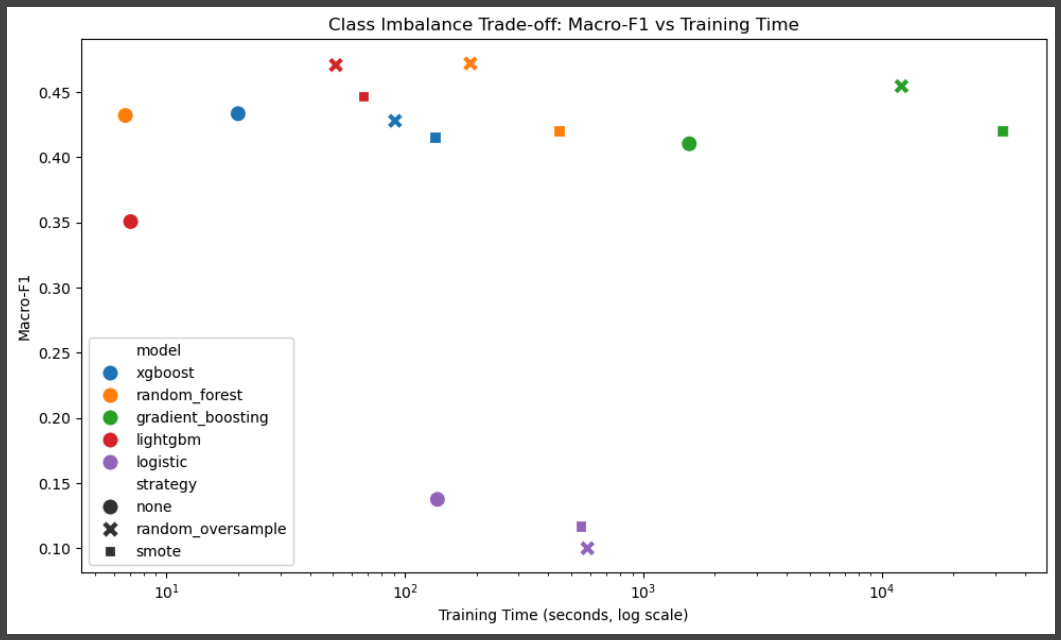
These 30 params are the ones we well use to build our model due to having the highest impact on the label. Now taking a closer look at the top 10, lets see if maybe we can find some unique characteristics for the attack types. Notable is that the DoS typically shows high packet rates and connection counts. Where exploits follow a pattern of specific protocol or packet size signatures. Backdoors have longer persistent connection patterns. Benign data has low flow packets, low amounts of forward packets, and also uses a reliably one port.

The Model:

The goal of this project was to create a model capable of detecting threats and what kind of threats they are. While modeling I chose a decision tree approach to processing the data. I compared XGBoost, Random Forest, Gradient Boosting, Logistic Regression, and LightGBM. Due to the nature of the data and implication of speedy response times I started with a benchmark compared to accuracy in the smaller groups of data.

Model	Macro F1	Prediction Time (Seconds)
XGBoost	43.35%	0.724
Random Forest	43.21%	0.626
Gradient Boosting	41.04%	2.553
LightGBM	35.07%	0.718
Logistic Regression	13.76%	0.021

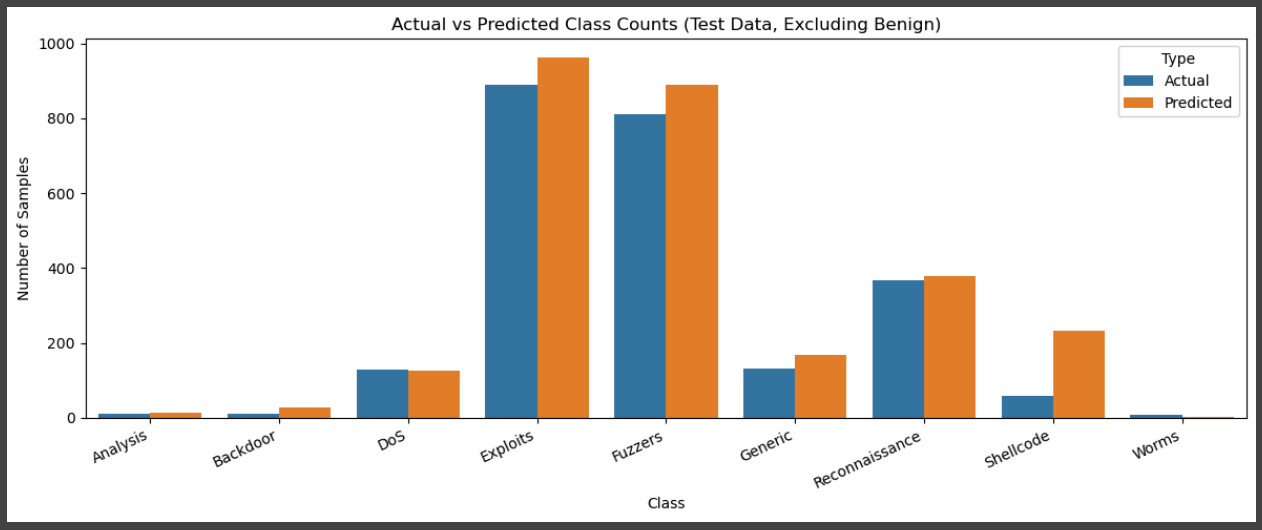
In these models untuned as they are. We can learn Logistic Regression is wildly inaccurate, also Gradient Boosting is very slow compared to the others. Naturally our data has lots of benign web traffic, as more activity is not an attack in real world scenarios. Making the data really skewed to Benign data, to combat this problem I tested a couple of ways to help balance the classes against each other. Oversampling and SMOTE.

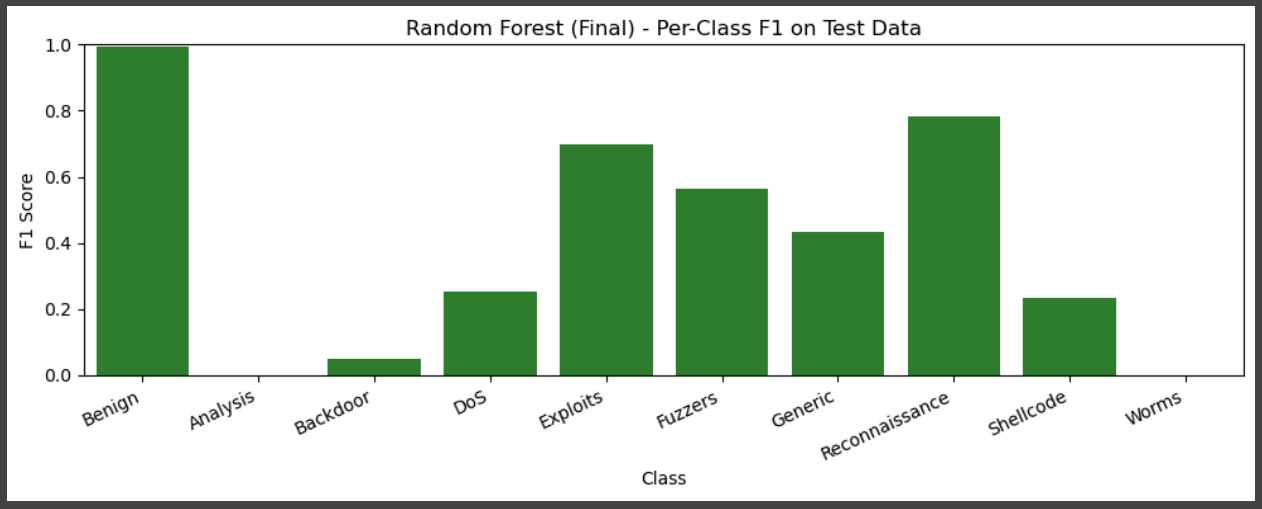


Looking at the scatter plot above, I am comparing the time it takes to create this model to how accurate it is at predicting the smaller categories. These are still untuned models, but we can see that LightGBM and Random Forest are the best when they are randomly over sampled. This is a process of taking the lesser known categories and duplicating them across the data set so that the model can see those less represented data more often. So from here I chose the top two performing models and did a random grid search to find the best parameters. Random Forest with random oversampling and LightGBM with random oversampling.

Comparing these two finalized models, after the grid search we get an average Micro F1 score of 95.85% for Random Forest and a score of 96.21% for the LightGBM. These are very close to each other and very similar in response speed. When looking at the predictions compared to the actual we get a better picture of what's happening under the hood. I will also be comparing the categorical micro f1 scores to get a better understanding of what classes the model is having a hard time predicting.

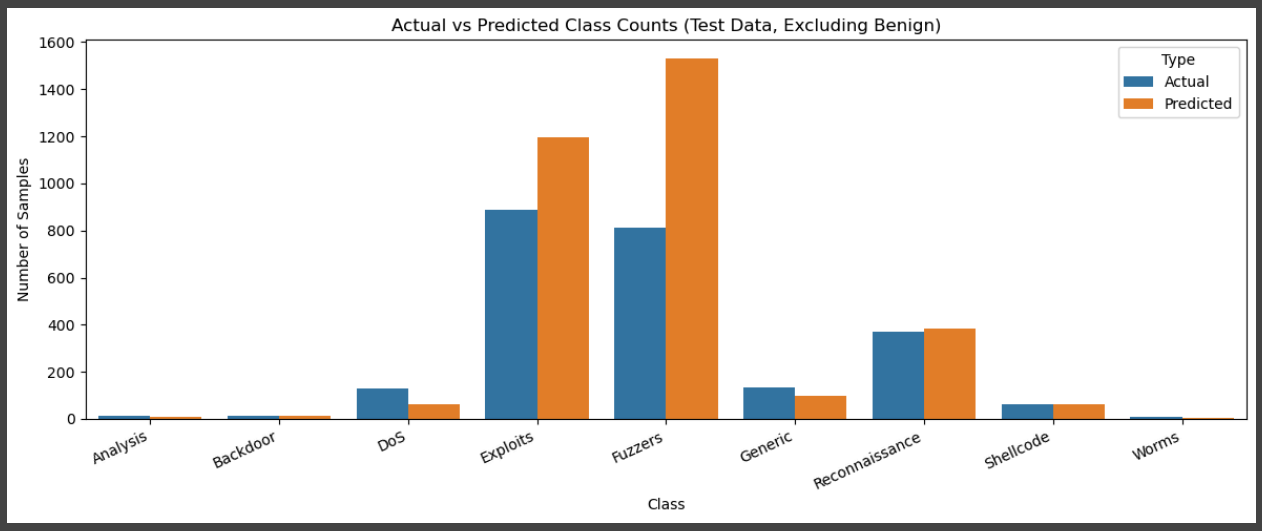
Random Forest:

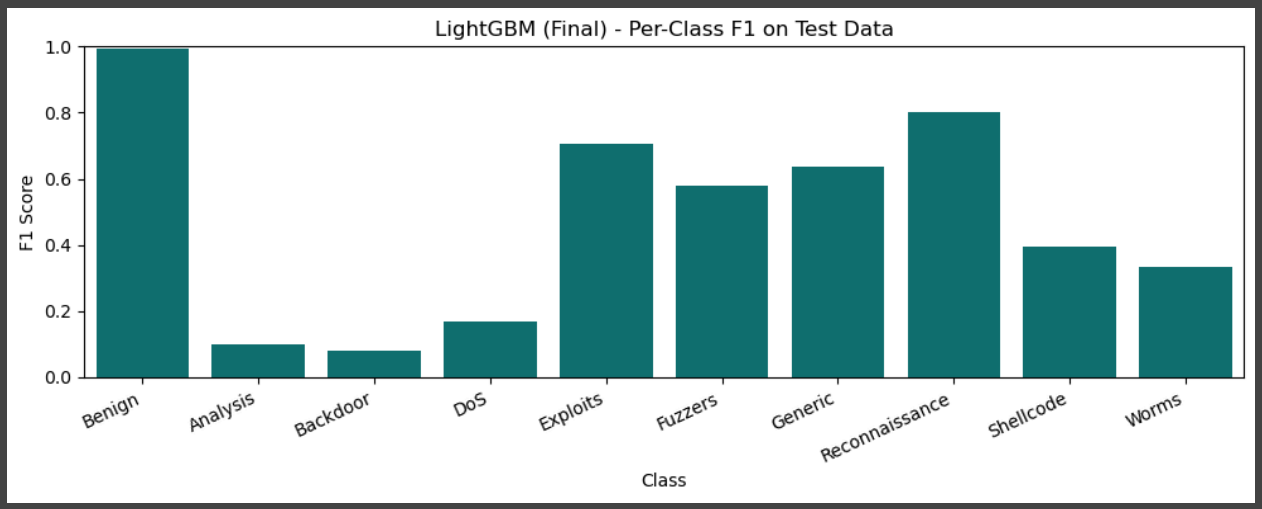




Looking at random forest, we can see that the guesses are fairly close to the actual with the expectation of shellcode. While looking at the F1 score chart it has no F1 for analysis or worms. This isn't good in practice because that means the model just got lucky guesses for some of the actual, or misclassified them.

LightGBM:

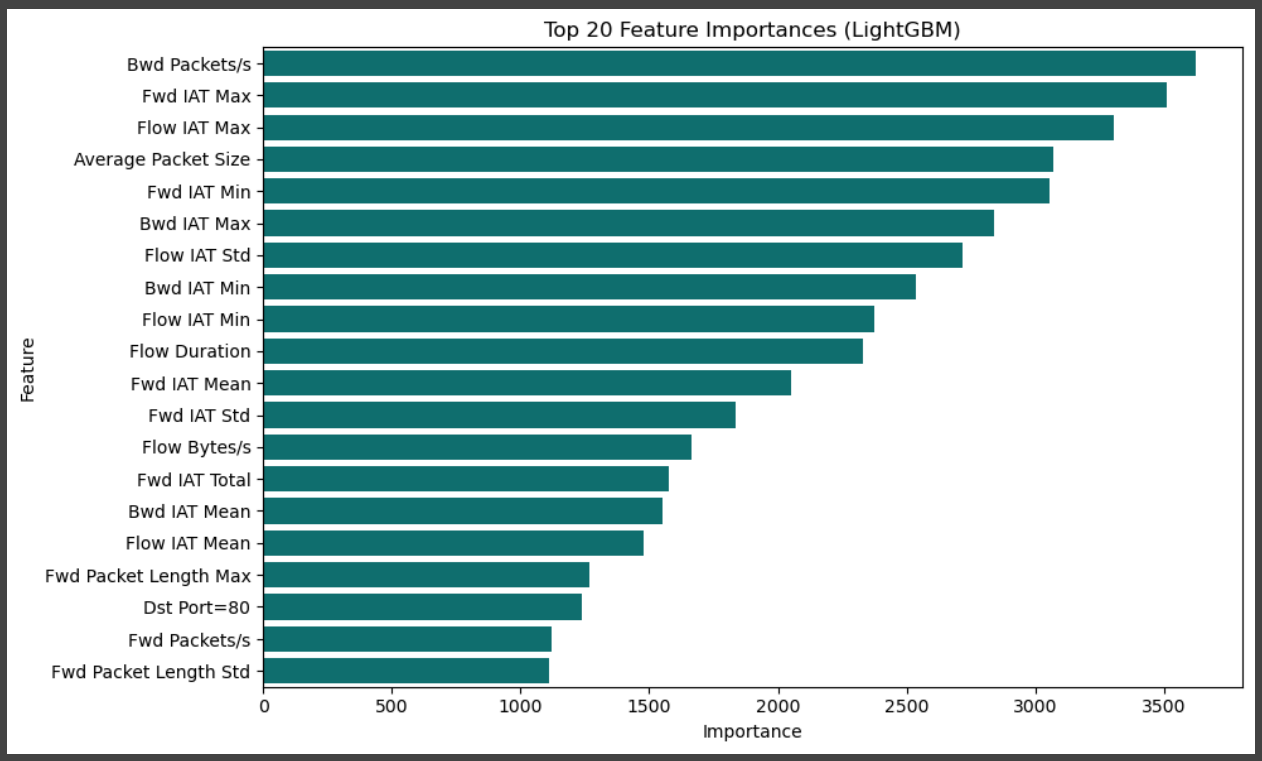




Now LightGBM, the guesses for Fuzzers has a lot of false positives as well as Exploits, but this comes from the nature of the data. Fuzzers and DoS, appear very similar though the data as both has similar attack patterns. Looking at the F1 Scores we get a better picture of what's happening. Unlike Random Forest we have no missing F1 scores. In this case some is better than none. So the LightGBM is 10% accurate at telling if analysis is happening on the network.

LightGBM:

With an average Macro F1 of 48% and 98% accuracy, I choose this model. Due to the fact that all the categories are represented to some extent. Now this is still a work in progress so I am hoping I can make a more accurate model at another time. But, for now let's look at some things I found interesting about this model.



This graph shows the rating or how important several of the features of this data is for making a production. This information will help me greatly as I continue to improve upon this model. Bwd Packet/s is how many packets are sent after a request has been made.

Thank You:

In the end the model I have for you is trying to balance speed and accuracy to eventually be made into a Network Scanner to help people stay safe on the internet. LightGBM, was the best choice I had at the time and look forward to increasing the capabilities of this model.

Citation:

H. Mohammadian, A. H. Lashkari, A. Ghorbani. “Poisoning and Evasion: Deep Learning-Based NIDS under Adversarial Attacks,” 21st Annual International Conference on Privacy, Security and Trust (PST), 2024.